Лабораторная работа 7 Сервис предсказаний и контракт API

Методические материалы курса · учебный год 2026–2027

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с индивидуальной сдачей Предшествующая работа: ЛР6 «Контейнеризация и управление средой»

1 Цель

Сформировать явный и тестируемый HTTP-контракт сервиса предсказаний: типизированные запросы и ответы, документированные ошибки, OpenAPI-схему, request ID, readiness и набор положительных и отрицательных contract tests.

2 Результаты обучения

Каждый студент должен уметь:

- отличать API-контракт от внутренней реализации модели;
- описывать request и response через Pydantic;
- запрещать неизвестные поля и нежелательное приведение типов;
- читать OpenAPI и связывать JSON Schema с тестом;
- различать 404, 422, 500 и 503;
- обеспечивать request ID в успешных и ошибочных ответах;
- тестировать API через TestClient и реальный HTTP-контур;
- объяснять совместимость изменения контракта;
- диагностировать назначенный негативный сценарий;
- подтверждать личный вклад независимо от результата команды.

3 Командный результат

Команда предоставляет tag lr7-v1 и:

1. FastAPI-сервис версии 0.3.0;
2. строгую PredictionRequest без неизвестных полей;
3. типизированную PredictionResponse;
4. единый ErrorResponse для 422 и 503;
5. X-Request-ID в header и request_id в body;
6. /healthz, /readyz, /predict, /metrics, /openapi.json и /docs;
7. сохранённую схему reports/openapi/lr7-openapi.json;
8. pytest contract tests OpenAPI, входов, выходов и ошибок;
9. HTTP-отчёт reports/api/lr7-contract-smoke.json минимум с четырьмя cases;
10. таблицу совместимости изменений;
11. issue, небольшие PR, review и индивидуальные evidence cards.

Сервис принимает только признаки, доступные за 60 минут до планового отправления. Фактическая задержка и другие поля будущего не могут быть частью request.

4 Контракт endpoints

Метод и путь	Назначение	Успех	Основные ошибки
GET /healthz	процесс отвечает	200	5xx инфраструктуры
GET /readyz	модель загружена	200	503
POST /predict	получить прогноз	200	422, 503
GET /metrics	технические метрики	200	5xx
GET /openapi.json	машиночитаемый контракт	200	5xx
GET /docs	интерактивная документация	200	5xx

/healthz не выполняет модельный прогноз. /readyz не подменяет contract test /predict.

5 Request schema

```
{
  "dep_hour": 9,
  "is_weekend": 0
}
```

Требования:

- dep_hour — integer от 0 до 23;
- is_weekend — integer 0 или 1;
- оба поля обязательны;
- строки "9", boolean и float не преобразуются неявно в integer;
- неизвестные поля запрещены;
- actual_delay, ArrDelay и иные признаки будущего отвергаются.

Строгость является решением контракта. Если клиенту нужна допустимая конверсия, она вводится явно, документируется и тестируется.

6 Response schema

Успешный ответ:

```
{
  "delay_probability": 0.37,
  "delayed": false,
  "threshold": 0.5,
  "model_version": "0.2.0",
  "request_id": "lr7-demo"
}
```

Инварианты:

- delay_probability находится в [0, 1];
- delayed == (delay_probability >= threshold);
- threshold явно присутствует;
- model_version соответствует bundle;
- request_id непустой и совпадает с X-Request-ID.

Response model ограничивает и документирует выход. Не возвращайте внутренний объект модели, stack trace, путь файловой системы или входные секреты.

7 Error schema

```
{
  "error_code": "validation_error",
  "message": "[redacted]",
  "request_id": "lr7-demo",
  "details": [
    {
      "location": ["body", "dep_hour"],
      "message": "Input should be less than or equal to 23",
      "error_type": "less_than_equal"
    }
  ]
}
```

Ошибка должна помогать клиенту исправить запрос, но не раскрывать stack trace или данные других запросов. Текст Pydantic может меняться между версиями; тестируйте стабильные поля error_code, status, location и наличие диагностической детали, а не полный текст сообщения.

8 Семантика status codes

Status	Значение в ЛР7
200	запрос выполнен по контракту
404	endpoint не существует
422	JSON получен, но не соответствует schema
500	необработанная внутренняя ошибка; считается дефектом
503	процесс жив, но модель не готова

Не возвращайте 200 с полем "error". HTTP status является частью машинного контракта.

9 Request ID

Middleware принимает непустой X-Request-ID разумной длины или создаёт UUID. Значение возвращается в response header. Для JSON-ответов /predict и ошибок оно также включается в body.

Request ID нужен для связи клиентского симптома, лога и метрики. Он не должен содержать персональные данные, token или полный payload.

Проверка:

```
curl -i -X POST http://127.0.0.1:8080/predict \
  -H 'Content-Type: application/json' \
  -H 'X-Request-ID: student-case-1' \
  -d '{"dep_hour":9,"is_weekend":0}'
```

10 Lifespan модели

Модель загружается один раз в lifespan перед обслуживанием запросов, а не на каждый /predict. Это сокращает latency и задаёт ясный readiness contract.

Если файл модели отсутствует, процесс может отвечать на /healthz, но /readyz и /predict возвращают 503. Если файл существует, но повреждён, startup должен завершиться наблюдаемой ошибкой, а не принимать запросы с частично инициализированным состоянием.

11 OpenAPI как исполняемая спецификация

Сохраните схему:

```
python scripts/export_openapi.py --output reports/openapi/lr7-openapi.json
```

Проверьте:

- info.version равен 0.3.0;
- /predict содержит POST;
- request body ссылается на PredictionRequest;
- additionalProperties равно false;

- оба поля перечислены в required;
- 200, 422 и 503 имеют JSON schema;
- response ссылается на PredictionResponse;
- error responses ссылаются на ErrorResponse.

Схема, сгенерированная из незакоммиченного кода, не является доказательством принимаемой версии.

12 Unit contract tests

Обязательные pytest cases:

1. корректный запрос возвращает 200 и полный response;
2. dep_hour=24 возвращает 422;
3. отсутствующее поле возвращает 422;
4. неизвестное поле возвращает 422;
5. строка вместо integer возвращает 422;
6. при отсутствующей модели /readyz и /predict возвращают 503;
7. request ID совпадает в header и body;
8. OpenAPI содержит request, response и error schemas.

Для отдельных значений используйте параметризацию. Тест не должен зависеть от запущенного Docker container; он использует TestClient и временную модель.

13 HTTP contract smoke

После запуска Compose:

```
python scripts/api_contract_smoke.py \
  --base-url http://127.0.0.1:8080 \
  --output reports/api/lr7-contract-smoke.json
```

Скрипт выполняет cases:

- valid — 200;
- hour_out_of_range — 422;
- unknown_field — 422;
- wrong_type — 422.

Каждый результат содержит request, ожидаемый и фактический status, response и X-Request-ID. Отчёт создаётся автоматически и не редактируется вручную.

14 Проверка OpenAPI в Swagger UI

Откройте /docs и проверьте пример request, обязательность полей, диапазоны и перечисленные responses. Затем сравните с lr7-openapi.json.

Swagger UI помогает исследовать контракт, но скриншот не заменяет сохранённую schema и автоматические tests. В production документация может быть закрыта политикой доступа, однако сам контракт остаётся доступен pipeline.

15 Совместимость изменений

Изменение	Обычно совместимо?	Действие
добавить необязательное поле ответа	да, если клиент игнорирует неизвестное	обновить schema и tests
удалить поле ответа	нет	новая версия API или миграция
сузить допустимый диапазон	нет для прежних клиентов	согласовать переход
добавить обязательное поле request	нет	версионирование и период миграции
добавить новый endpoint	да	документировать
изменить смысл существующего поля	нет	новое имя или версия

Совместимость определяется поведением клиентов, а не только тем, что сервер запускается.

16 Негативные сценарии

Каждый участник воспроизводит один контролируемый отказ:

- разрешить неизвестные поля;
- убрать strict mode и принять строку "9";
- удалить response model;
- перестать возвращать request ID в header;
- сделать /readyz успешным без модели;
- убрать документированный response 503 из OpenAPI.

До изменения студент называет тест, который должен упасть. Дефект не входит в tag lr7-v1.

17 Таблица рисков

Риск	Контракт	Проверка
клиент отправляет признак будущего	extra forbid	unknown_field case возвращает 422
тип преобразован молча	strict input	строка вместо integer возвращает 422
ответ изменился незаметно	response model и OpenAPI	schema test
модель не загружена	readiness	503 и error_code not_ready
запрос нельзя найти в логах	request ID	header равен body
клиент зависит от текста ошибки	stable error_code	tests не сравнивают весь message

18 Индивидуальная сдача участника команды

Командный результат: tag lr7-v1, OpenAPI JSON, успешный pytest и HTTP contract-smoke.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР7, commit/tag и image ID ЛР6/ЛР7
Роль	роль в ЛР7 и отличие от ЛР6
Задача	issue с наблюдаемым критерием приёмки

Поле	Что указать
Личный артефакт	model schema, endpoint, test, smoke case, review или OpenAPI-проверка
Вклад	созданное, исправленное или проверенное изменение
Инженерное решение	выбранная строгость или форма контракта и альтернатива
Риск	несовместимость или отказ, обнаруживаемый тестом
Диагностика	status, request ID, test node ID и причина
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: командная версия зафиксирована, OpenAPI соответствует commit, личная issue закрыта через PR, review выполнен другим участником, tests зелёные, личный negative case воспроизводится.

19 Защита

За 3–4 минуты участник:

1. показывает свой элемент OpenAPI или код контракта;
2. выполняет положительный запрос;
3. выполняет назначенный отрицательный запрос;
4. связывает status, error_code и request ID;
5. находит проверяющий test node ID;
6. объясняет совместимость одного изменения;
7. восстанавливает корректную версию после defect injection.

Преподаватель может добавить неизвестное поле, заменить integer строкой, убрать модель или попросить найти schema конкретного response.

20 Повторная сдача и стоп-факторы

Повторная сдача выполняется через issue, отдельный PR, обновлённую OpenAPI schema, новый smoke report и evidence card. История первой попытки сохраняется.

Стоп-факторы:

- endpoint принимает произвольный dict вместо типизированной модели;
- поле будущего принимается без ошибки;
- error возвращается со status 200;
- /readyz успешен без модели;
- request ID отсутствует или содержит секрет;
- schema не соответствует принимаемому commit;
- negative tests отсутствуют или всегда зелёные;
- студент не может объяснить собственное поле или status;
- отчёт изменён вручную.

21 Критерии оценивания

Критерий	Баллы	Полное выполнение
Request contract	2	типы, диапазоны, strict и extra forbid
Response и errors	2	типизированный ответ, error model и status semantics
OpenAPI	2	schema сохранена и автоматически проверена
Request ID и readiness	1	header/body согласованы, 503 воспроизводим
Командный процесс	1	issue, небольшие PR и содержательное review
Индивидуальная защита	2	личный артефакт, negative case и совместимость

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

22 Чек-лист

- [ ] Команда состоит из 2–5 человек, роли ротированы.
- [ ] Request и response описаны Pydantic-моделями.
- [ ] Неизвестные поля и неявные типы отвергаются.
- [ ] 422 и 503 имеют единый ErrorResponse.
- [ ] Request ID совпадает в header и body.
- [ ] /readyz зависит от загруженной модели.
- [ ] OpenAPI сохранён из принимаемого commit.
- [ ] Pytest содержит положительные и отрицательные cases.
- [ ] HTTP smoke report содержит минимум четыре cases.
- [ ] Каждый участник выполнил defect injection и диагностику.
- [ ] PR прошёл review, создан tag lr7-v1.

23 Связь с ЛР8

В ЛР8 Ruff, pytest, data validation, OpenAPI export и container build станут обязательными quality gates GitHub Actions. Pull request не может быть принят, если contract tests или проверка schema завершились неуспешно.

24 Официальные источники

- FastAPI request body and models: https://fastapi.tiangolo.com/tutorial/body/
- FastAPI response models: https://fastapi.tiangolo.com/tutorial/response-model/
- FastAPI error handling: https://fastapi.tiangolo.com/tutorial/handling-errors/
- FastAPI lifespan: https://fastapi.tiangolo.com/advanced/events/
- FastAPI OpenAPI: https://fastapi.tiangolo.com/tutorial/metadata/
- Pydantic models and configuration: https://docs.pydantic.dev/latest/concepts/models/